

module 2

Stat 5200: Analysis of Designed Experiments

Jesse Wheeler

Overview

Objectives and overview

- This material is based Chapter 2 on the textbook “A First Course in Design and Analysis of Experiments” (Oehlert, 2000).
- Chapter outcomes include:
 - Understanding the importance of randomization to mitigate confounding and bias.
 - Understanding the important role randomization plays in statistical inference.
 - Reviewing basic inference concepts, such as hypothesis testing and basic t -procedures.

Randomization

Randomization and Experiments

- An experiment is called randomized when the assignment method to units involves explicit use of an *understood* probability mechanism.
- Proper randomization is essential to ensure that uncontrolled factors do not introduce systematic bias in your results.
- Randomization is not “haphazard” assignment.
- E.g.: Just assigning treatments sequentially to the next experimental unit does *not* count as random, even if it appears to you there is no pattern among the units. **Consider the following assignment methods:**

(Next slide)

Randomization and Experiments II

In all cases, suppose that we have four treatments that need to be assigned to 16 units.

- (i) Use 16 slips of paper marked with letters, 4 A 's, 4 B 's, 4 C 's, and 4 D 's. You mix them thoroughly in a box or basket. For each EU, you pick a slip and assign it the treatment indicated.
- (ii) You assign A to the first 4 units you have at hand, B to the next 4, C to the next 4, and D to the last 4.
- (iii) If, when you get to an EU, the second hand is between 1 and 15, the unit gets A ; if between 16 and 30, it gets B ; if between 31 and 45, it gets C ; and if between 46 and 60, it gets D .

1.5 min

$$\begin{aligned} \rightarrow P(A) &= \frac{30}{90} = \frac{1}{3} \\ \rightarrow P(B) &= \frac{30}{90} = \frac{1}{3} \end{aligned}$$

$$P(C) = P(D) = \frac{15}{90} = \frac{1}{6}$$

Why Randomize?

- “Introducing more randomness into an experiment may seem like a perverse thing to do”, however it has two useful consequences (Oehlert, 2000):

- ✱ 1. Randomization protects against **confounding**. ✓
- ➔ 2. Randomization can form the basis for inference.

The course focuses on the impact on confounding.

Why Randomize? II

Example: Protecting against confounding.

Confounding occurs when the effect of one factor or treatment cannot be distinguished from that of another factor or treatment.

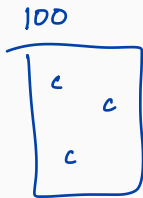
EX: In a medical study, we wish to see if treatment A (costly and invasive) or treatment B (standard drug treatment) results in improved health outcomes. We have 100 volunteers, each needs to be assigned a treatment (A or B).

Q: What may “go wrong” if we don’t randomize assignment (e.g., let the physician pick).

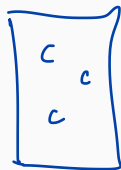
- Different treatment group sizes.
- Bias from choice.
- Patient Demography.

Why Randomize? III

- If the assignment is truly randomized, then any factors other than the ones included in the treatments (i.e., lurking variables) will tend to average out so they affect all treatments equally and have little net effect.



A



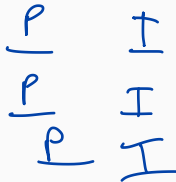
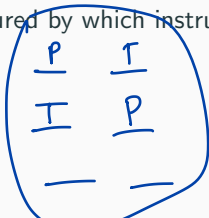
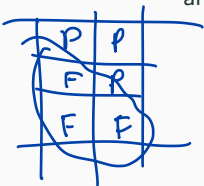
B



- Randomization generally costs little in time and effort, but can greatly enhance your analysis and save us from disaster.

Randomizing “Other Things”

- We have taken a simple view so far of experiments: “assign treatments to units, and measure responses”.
- Though naturally we do assignments randomly, additional things can and should be randomized:
 - If the EUs are not used simultaneously, you can randomize the order in which they are used.
 - If the EUs are not used at the same location, you can randomize the locations at which they are used.
 - If you use more than one measuring instrument (e.g., multiple machines or undergraduates), you can randomize which units are measured by which instrument.



Randomizing “Other Things” II

- Whenever we anticipate that some way the experiment is conducted or measured may potentially impact the response, we can **design** that into the experiment (more to come later).
- **Advice:** Design for known problems, randomize everything else.

Randomization Methods

Randomization methods and examples

- Randomization methods largely fall into two categories: physical and numerical.
- For modern designs and experiments, we generally use numeric approaches like a pseudorandom number generator.]
- A few examples (2 treatments, A and B). What are some potential advantages or disadvantages of each?

Randomization methods and examples II

- (i) Flip a coin for each patient, heads give treatment A , tails give treatment B .

Advantages

- understood randomization
- simple

Disadvantages

- treatment groups may not be same size.

- (ii) Take patients in pairs, perhaps approximately matched by demographic or physical variables, flipping a coin to choose which gets A , then giving B to the other patient.

Advantages

- control for demog.
- equal size groups

Disadvantages

- large sample size
- matching is hard

- ★ (iii) If you have 30 patients, say select a random sample (without replacement) of size 15 to get A and give B to the remaining 15 patients.

Advantage

- simple
- balanced groups.

Disadvantage

- for small groups, not average effects.

Randomization methods and examples III

- The analysis we perform depends on how we randomize.

Computer Randomization

- In 2026, our primary means of randomization will be using computers and **pseudorandom** number generators.
- One of the most important functions we will seed is `set.seed`, which allows our randomization to be **reproducible**.

`set.seed(123)`

1 2 3 4 5 6

- Pick any positive integer. Avoid “Seed hacking”.

`set.seed(123)`
`sample(c("H", "T"))`
"H", "T"

Computer Randomization II

- Another key randomization function is `sample`. This is a flexible method that can be used in a variety of ways.

4 arguments:

```
sample(x, size, replace, prob)
```

- `x`: Any 'R' object, typically a vector (character or numeric).
The “things” we want to randomly draw.
- `size`: How many “things” in `x` that we want to sample.
Default is the same size as `x`.
- `replace`: Do we want sampling with (`TRUE`) or without (`FALSE`, default) replacement.
- `prob`: Do we want to have un-even sampling probabilities (e.g., biased coin?)

Computer Randomization III

- **EX:** Suppose you have $N = 31$ EU's that you want to assign to 4 treatments, with $n_A = n_B = n_C = 7$ and $n_D = 10$.
- One way to do this physically: create slips of paper labelled $1, 2, \dots, 31$, put them in a box, shuffle well, and draw numbers sequentially. The first 7 will be assigned treatment A , the next 7 treatment B , etc.
- We can mimic this approach in R:

```
set.seed(1111111)
```

```
sample(1:31) # permute the numbers 1 -- 31.
```

```
[1] 28 4 9 23 1 30 11 31 12 15 22 14 19 16 7 25 21  
[18] 27 29 2 17 8 5 3 13 6 20 26 18 24 10
```

$1:31 = \dots 31$

Handwritten annotations on the R output:

- Blue brackets group the first 7 numbers (28, 4, 9, 23, 1, 30, 11) and the next 7 numbers (31, 12, 15, 22, 14, 19, 16).
- Blue brackets group the last 7 numbers (7, 25, 21, 27, 29, 2, 17) and the remaining 7 numbers (8, 5, 3, 13, 6, 20, 26).
- Blue brackets group the last 10 numbers (8, 5, 3, 13, 6, 20, 26, 18, 24, 10).
- Handwritten labels A , B , C , and D are placed near the corresponding groups.

Computer Randomization IV

- Above, we're permuting the numbers (EUs), and then "labeling" them sequentially with treatment.
- An alternative approach can also be used to permute the treatments, and then sequentially assign EUs.

```
set.seed(1111111)
treatments <- c(
  rep("A", 7), rep("B", 7),
  rep("C", 7), rep("D", 10)
)
sample(treatments)
```

A, A, A, A, A, A, A, B, B, ...,
C, C, ..., C,
D, ..., D.

```
[1] "D" "A" "B" "D" "A" "D" "B" "D" "B" "C" "D" "B"
[13] "C" "C" "A" "D" "C" "D" "D" "A" "C" "B" "A" "A"
[25] "B" "A" "C" "D" "C" "D" "B"
```

Computer Randomization V

- **EX:** Randomizing matched pairs. Assume that we have 30 workers, and two treatments (A and B). We want to give each treatment to each worker, but in random order.
 - Physically, we can use 30 coin flips. For the n th flip, if heads, assign treatment A first, B otherwise.
 - In R:

```
set.seed(222222)
```

```
# "prob" not set, each outcome equally likely
```

```
sample(c("A", "B"), size = 30, replace = TRUE)
```

```
[1] "A" "A" "B" "A" "B" "B" "B" "A" "A" "A" "A" "B"  
[13] "A" "B" "A" "A" "A" "B" "B" "A" "A" "B" "A" "B"  
[25] "B" "A" "A" "A" "A" "B"
```



Computer Randomization VI

- If we instead want a “biased” coin-flip:

```
set.seed(222222)
# "probs" not set, each outcome equally likely
sample(
  c("H", "T"), size = 30,
  replace = TRUE, prob = c(0.7, 0.3)
)

[1] "H" "H" "T" "H" "H" "H" "H" "H" "H" "H" "H" "H"
[13] "H" "H" "H" "H" "T" "H" "H" "H" "T" "H" "T" "H"
[25] "H" "H" "H" "H" "T" "H"
```

ch 2 .

Randomization as basis for inference

Frequentist statistics

- Randomization also forms the basis of one form of statistical inference.

Review: Hypothesis testing and inference

Suppose we want to compare two competing ideas (hypotheses), and come up with data-driven conclusions.

We will call the prevailing standard idea the “Null Hypothesis” (H_0), and the alternative the “Alternative Hypothesis” (H_1).

Framework: Assume that H_0 is true; if so, how “frequent” is the observed outcome relative to all possible outcomes? If it is a rare outcome under H_0 , we assume H_0 is incorrect and favor H_1 .

Frequentist statistics II

- **EX:** Paired t -test for testing the effect of workplace conditions.

Randomization as a form of inference

- What was done: Create a null-hypothesis (means for both groups are the same), compare what we saw to the range of all possibilities. **Q:** Why the normal assumption?
- The key to the procedure above is assuming the observations are random. We'll use this logic to create a **randomization test**.
 - This is an *exact* statistical test, which doesn't require normality assumption!

Randomization as a form of inference II

Randomization Test

- Hypothesis: treatment has no effect. That is, the treatment can be thought of as a label, not something that would change outcomes.
- Pick a statistic of interest (i.e., the mean, median, etc.). Compute this for the observed data.
- Since treatments are just “labels”, and assuming we randomized these “labels”, compute the same statistic for *all* possible assignments.
- Compare statistics to get P-value: how abnormal is the actual data relative to all random assignment possibilities?

Randomization as a form of inference III

- *Advantage:* No normality assumption. No assumptions at all!
- *Disadvantage:* Can be extremely computationally intensive.

Randomization as a form of inference IV

Last 10 observations

Here are differences for the last 10 observations:

```
[1] 0.62 1.75 0.71 0.21 0.01 0.42 -0.21 0.42  
[9] 0.43 0.82
```

Randomization as a form of inference V

- Even with $n = 10$, we require computing $2^{10} = 1024$.
- Using software, we found that only $4/1024 \approx 0.004$ of the means were larger than what we observed.
- This approach becomes computationally burdensome as n grows.
- Compare this to the P-value of a t-test: 0.007.

Randomization as a form of inference VI

- Is there an easier way?
- Sometimes... We don't actually need to know the value of all possible permutations, only the percentage of them that are larger (or smaller) than or equal to what we saw.
- Our data: 9 positive values, 1 negative value (-.21). 7 of the 9 positive values has magnitude larger than 0.21. The remaining numbers are: +0.21, +0.01

Approximate tests

- Even with clever strategies, it's likely computationally unfeasible for even small n and two treatments.
- We generally therefore resort to approximate methods. For instance, it can be argued that the t -test is an approximate test.

Approximate test: random subsamples

Instead of computing 2^{30} possible combinations: randomly sample a subset (10,000) assignments, and compare to the observed data as you would normally.

Approximate tests II

For our $n = 30$ data set, here's example R code (from scratch) that could compute an approximate randomization test:

```
set.seed(123)
true_mean <- mean(data)
null_means <- replicate(10000, {
  random_signs <- sample(c(1, -1), size = 30, replace = TRUE)
  mean(data * random_signs)
})

# P-value
mean(null_means > true_mean)

[1] 0.0738
```


References and Acknowledgements

Oehlert GW (2000). *A First Course in Design and Analysis of Experiments*. W. H. Freeman. ISBN 0-7167-3510-5.

- Compiled on September 3, 2026 using R version 4.5.0.
- Licensed under the [Creative Commons Attribution-NonCommercial](#) license. Please share and remix non-commercially, mentioning its origin.
- We acknowledge [students and instructors for previous versions of this course / slides](#).

